

恶意代码分析与防治技术实验报告

R77 技术分析



网络空间安全学院

信息安全专业

2211985 李佳璐

一、实验目的

在使用R77的基础上，撰写技术分析，要求描述使用过程中看到的行为如何技术实现。

二、实验过程

1. 安装

部署 r77 只需要一个文件：**Install.exe**。执行将 r77 保留在系统上，并注入所有正在运行的进程。

Uninstall.exe 从系统中完全、优雅地删除 R77。

Install.shellcode 是安装程序的等效 shellcode。这样，可以在不掉落**Install.exe**的情况下集成安装。shellcode 可以简单地加载到内存中，强制转换为函数指针，然后执行：

```
int main()
{
    LPBYTE shellCode = ...//加载 shellcode

    //修改内存保护权限
    DWORD oldProtect;
    VirtualProtect(shellCode, shellCodeSize, PAGE_EXECUTE_READWRITE,
&oldProtect);
    //VirtualProtect 是 windows API, 用于更改某块内存区域的访问权限

    ((void(*)())shellCode)(); //执行 shellcode

    return 0;
}
```

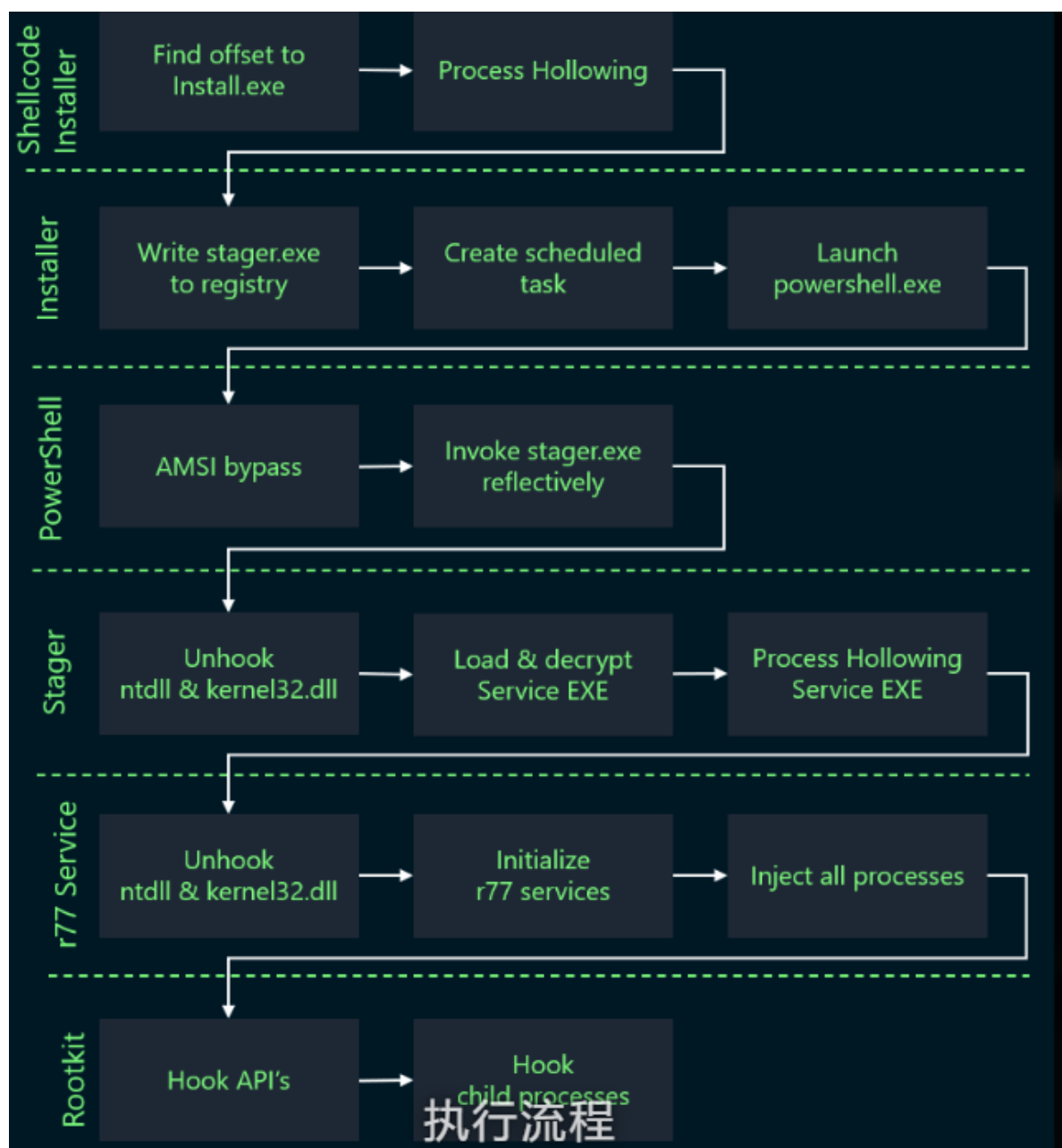
1. **加载 shellcode**：从程序资源或字节数组中加载 shellcode。
2. **修改内存保护**：通过 `VirtualProtect` 函数将 shellcode 区域的权限设置为可执行。
3. **执行 shellcode**：通过将内存区域转换为函数指针并调用它，实际上执行了 `Install.shellcode`，实现了 r77 的安装功能。

这种安装方式属于 **内存注入**，即 shellcode 完全在内存中运行，不需要写入磁盘。这可以避免传统的文件扫描检测，也更难被防病毒软件检测到，因为没有实际的磁盘文件存在。

2. 执行流程

rootkit 驻留在系统内存中，不会向磁盘写入任何文件。这是通过多个阶段实现的。

此图显示了从安装程序的执行一直到每个进程中运行的 rootkit DLL 的每个阶段。



整体查看该项目，分析知，其主要由四个独立模块组成：

- The Install Module.
- The stager module.
- The service module.
- The core module.

四个模块共同完成了隐藏任务。

The Install Module

1. 存储分阶段加载模块（Stager Module）：

- 将分阶段 Stager Module 的 PE 文件存储在注册表中，用于在系统中持续存储其恶意代码。

2. 构建 PowerShell 命令：

- 一旦将 Stager Module 存储在注册表中，Install Module 构建一个 PowerShell 命令，用于从注册表中加载并执行 Stager Module。然后，Install Module 创建一个定时任务，以运行该 PowerShell 命令。

3. 定位并存储 Stager Module：

- Install Module 定位了存储为 PE 资源命名为 EXE 的 Stager Module 。随后，在 HKEY_LOCAL_MACHINE\SOFTWARE 注册表 hive 中创建了一个名为 \$77stager 的新注册表键，并将 Stager Module 写入该键。

4. 反射性加载 .NET Stager Module:

- Install Module 构建了一个 PowerShell 命令，通过 [Reflection.Assembly]::Load 方法反射性地加载 .NET Stager Module 到内存中。

5. 绕过 AMSI (Antimalware Scan Interface) :

- Install Module 通过修改 AmsiScanBuffer API 绕过 Microsoft 的 AMSI，使其始终返回 AMSI_RESULT_CLEAN 响应。AMSI_RESULT_CLEAN 表示扫描的内容是“已知良好的，未发现检测，未来的定义更新后结果可能不会改变。”

6. 混淆 PowerShell 命令:

- 最后，通过将变量名替换为随机字符串，对 PowerShell 命令进行混淆，增加检测的难度。

7. 创建定时任务:

- Install Module 创建了一个使用 COM 对象执行 PowerShell 命令的定时任务，该任务配置为在系统启动时以 SYSTEM 账户身份执行。增加了 Rootkit 的持久性和在系统启动时的执行。

The Stager Module

该模块是一个基于 .NET 的二进制模块，负责取消 DLL 的 hook，调整 SeDebugPrivilege 设置，解密和解压缩 Service Module，并将 Service Module 注入。此过程通过使用 process hollowing 技术，采用 PPID spoofing 实现。其有如下作用：

- 允许在一开始使用较小的 payload 来加载更多功能的较大的 payload，让攻击手法更加隐蔽。
- 使通信能够和最终阶段分离，故而无需复制代码，一个 payload 就可以在不同途径传输多次。
- 由于 stager 已经为程序分配了大量内存，使得 stages 不需要考虑大小问题，可以任意大，进而 stages 能够以更高级别的语言编写最终阶段的 payload，并且动态进行加载。

The Service Module

该模块负责执行如配置注册表等的关键任务。

The Core Module

该部分为 Service Module 注入到进程中的 dll，主要负责调用 Detours 在重要的 API 上进行 hook，并根据其配置过滤输出。这些原生 API 可以用于列举或获取有关的系统信息，则安装的 hook 就可以列举并过滤出攻击者配置好的 key，其余的则转发给 regedit 进程。通过过滤操作，即可实现有选择地隐藏文件、进程抑或是注册表键值等信息。

3. AV/EDR evasion (防病毒/端点检测与响应逃避) 技术

这些技术的目标是让恶意软件在执行时避开传统的安全防护机制，特别是在恶意软件活动被监控的情况下。

AMSI 绕过: 通过修改 `AmsiScanBuffer` 函数，使得任何通过 AMSI 扫描的脚本或代码总是被标记为干净，避免被安全软件检测到。攻击者利用多态性来动态修改脚本内容，绕过签名检测。

DLL 去钩: 攻击者通过加载干净版本的 `ntdll.dll`，去除 EDR 解决方案所添加的 API 钩子，这样可以防止恶意代码被检测到。避免了 EDR 通过 API 钩子检测到进程注入或其他恶意行为。

(1) AMSI Bypass (绕过 AMSI)

AMSI (Windows 防恶意软件扫描接口) 是 Windows 中的一个防护机制，用来扫描和检测脚本（如 PowerShell 脚本）中可能存在的恶意内容。许多安全软件都使用 AMSI 来扫描脚本执行时的内容。

- **绕过方式：**

攻击者通过 **修改** `amsi.dll` 中的 `AmsiScanBuffer` 函数，强制其始终返回

`AMSI_RESULT_CLEAN`（表示“没有问题”）。这会使得 AMSI 无法检测到恶意脚本或代码，攻击者通过这种方式避开了 AMSI 的监控。

- **技术实现：**

攻击者通常会使用 **内联脚本**（例如 PowerShell 脚本）来执行这个操作，直接在脚本中通过修改内存或者调用特定的系统 API 来绕过 AMSI 的扫描机制。这种技术往往使用 **多态性**（polymorphism），即动态修改脚本内容，避免被静态签名检测到。例如，脚本中的函数或代码会根据每次执行时的不同而变化，从而避免被传统的签名识别。

多态性 使得每次绕过操作的代码会略有不同，使得 AV/EDR 系统无法通过已知的特征码来进行检测。

(2) DLL Unhooking (DLL 去钩)

EDR (端点检测响应) 解决方案 通常会通过 **API hooking**（钩取 API）来监控进程的行为。这意味着它们会修改系统的 API 函数（例如 `ntdll.dll` 中的函数）以便拦截和分析进程的调用，通常用于检测恶意行为（如进程注入、代码注入、系统调用等）。

- **去钩方式：**

为了绕过这些监控，攻击者可能会通过 **加载新的** `ntdll.dll`（从磁盘读取并加载一个干净的版本），并恢复原始的内存结构。这样，安全监控工具在接下来的分析过程中，就不会看到由 EDR 或 AV 工具添加的钩子。

- **API Hooking**：通常，EDR 会通过修改函数指针来拦截并监控 API 调用，例如，钩住 `ntdll.dll` 中的 `NtCreateThread` 或 `NtQueueApcThread` 等函数。这些 API 钩子会在函数执行之前或之后执行额外的检查，试图捕获恶意行为。
- **恢复原始 API**：攻击者通过加载一个干净的 `ntdll.dll` 副本并将其放置在内存中，替换原本的 `ntdll.dll`，从而移除这些安全钩子。这样，恶意代码的行为就不会被 EDR 或 AV 系统的钩子检测到。

- **过程注入 (Process Hollowing)：**

另一种常见的逃避方法是 **过程空洞** (Process Hollowing)，即攻击者将恶意代码注入到一个正常的进程中。在注入过程中，攻击者需要绕过钩子检测，确保自己修改的代码不会被 EDR 监控到，因此去钩的技术非常重要。如果 EDR 监控到了 API 钩子的变化，它可能会发现恶意注入行为。

三、使用 R77 过程中的行为技术具体分析

1. 前缀隐藏

指导文档中给出，R77定义了一个"\$77"隐藏前缀

2.6 Hide Prefix

```
#define HIDE_PREFIX L"$77"
```

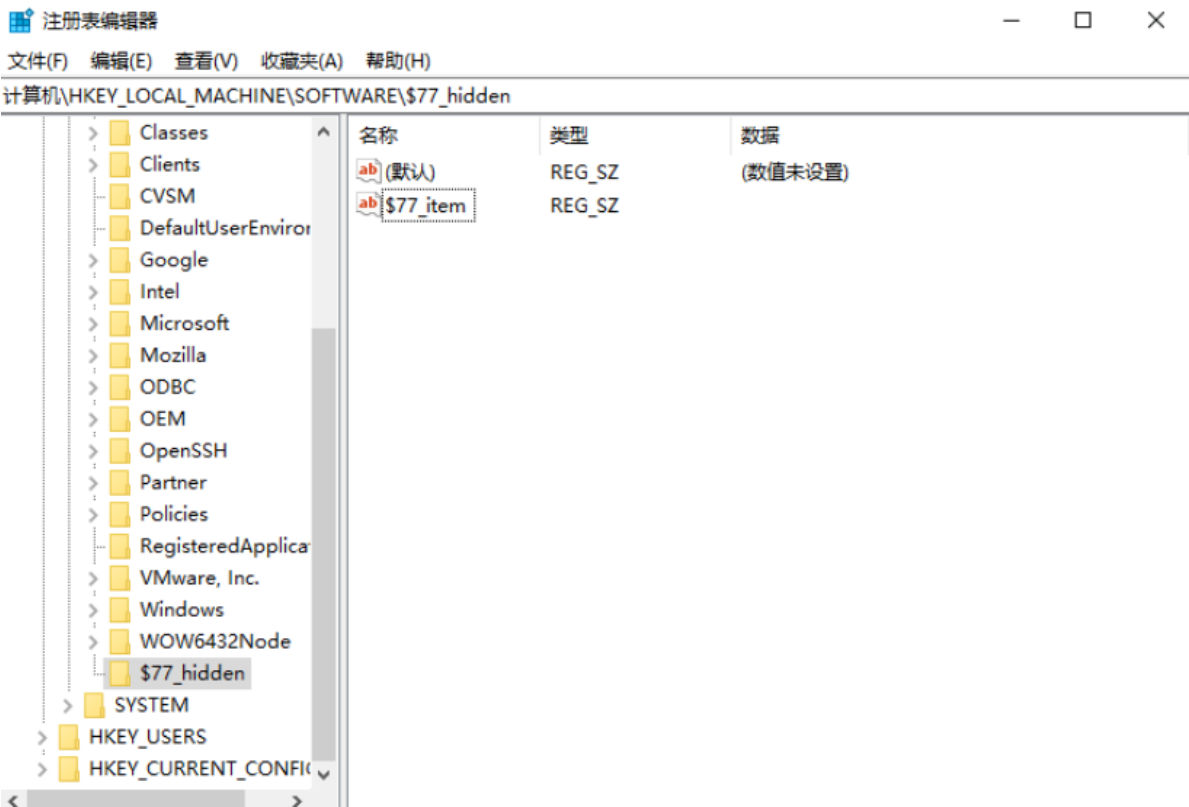
The hide prefix is a compile time variable. It specifies the beginning of a name that is hidden. This applies to the filenames of processes, to files and directories, and all other entities described in section 2.5. If the documentation mentions the string \$77, it refers to the compile time variable that is used throughout the code. This is the single place to change the prefix. Additionally, this change must be applied in GlobalAssemblyInfo.cs.

Processes that are hidden by prefix cannot be injected with r77. The DllMain of r77 will return FALSE, if the process starts with the prefix.

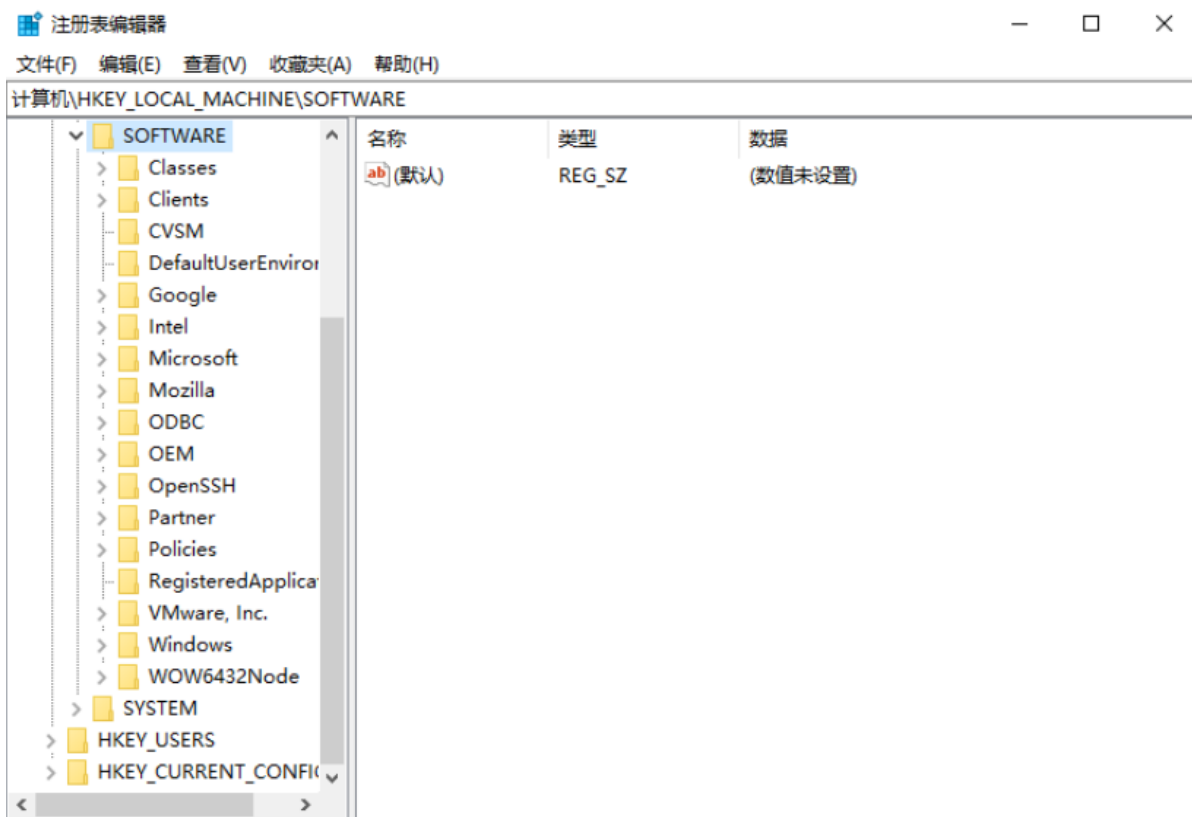
隐藏前缀是一个编译时的变量。它指定了一个被隐藏的名称的开头。这适用于**进程的文件名、文件和目录**，以及**文件、注册表和网络连接**的所有其他实体。如果以字符串\$77开头，即会被隐藏。

eg.

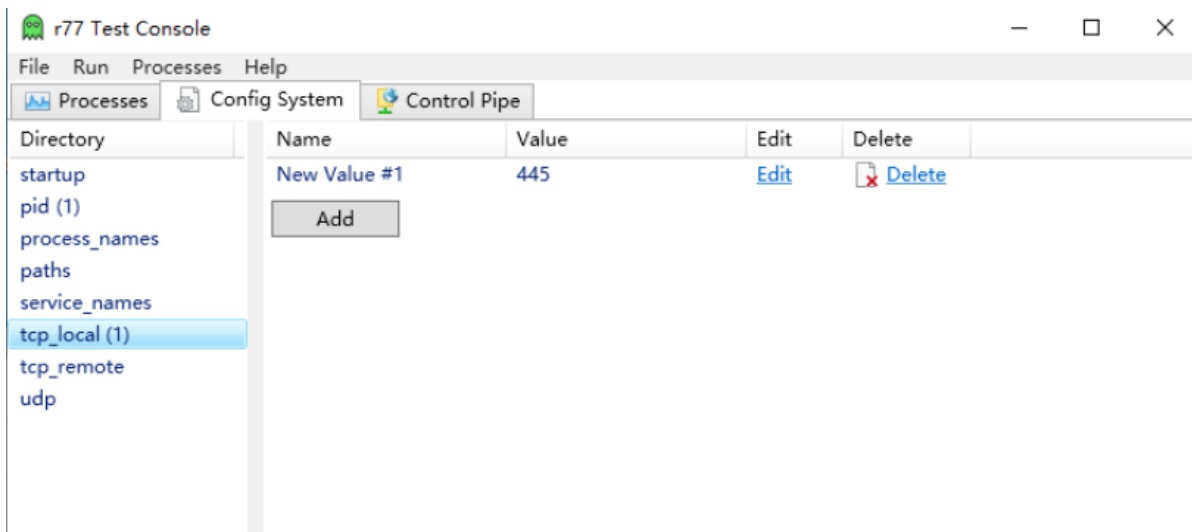
隐藏前:



隐藏后:



2. 使用配置系统添加要监控隐藏的进程或文件等



1. 进程隐藏

r77 通过挂钩 (hooking) 系统 API 实现对进程的隐藏。它拦截 Windows API 函数 (例如 `NtQuerySystemInformation`)，修改返回的进程列表，将特定的进程从列表中移除。这样，使用系统任务管理器或者其他进程监视工具时，看不到被隐藏的进程。

2. 文件隐藏

r77 也会挂钩文件系统的相关 API (例如 `NtQueryDirectoryFile`)，从文件列表中移除特定的文件或文件夹。它可以识别特定文件名或路径，然后修改返回的结果，以实现指定文件或文件夹的隐藏。

3. 注册表隐藏

对于注册表项的隐藏，R77 通过挂钩注册表相关的 API（例如 `NtEnumerateKey` 和 `NtQueryKey`），过滤掉特定的注册表键或值，使其在注册表编辑器等工具中不可见。这同样是通过拦截 API 返回的数据并过滤出指定内容来实现的。

4. 网络连接隐藏

R77 使用类似的技术来隐藏网络连接。通过挂钩网络相关的 API 或操作系统接口（例如 `NtDeviceIoControlFile`）实现对特定端口、协议或 IP 地址的隐藏。它可能拦截系统的网络连接查询请求，从结果中删除特定的连接信息。

3. 使用过程的具体技术实现

（1）进程隐藏

行为表现：当 R77 被注入到目标系统中时，用户运行任务管理器（如 Windows 的 Task Manager）时，并不会看到特定的恶意进程或注入的程序。这是因为 R77 会通过修改系统的进程信息，来达到隐藏进程的效果。

技术实现：R77 通过挂钩操作系统的进程查询 API（如 `NtQuerySystemInformation` 或 `NtQueryInformationProcess`）来实现进程隐藏。当应用程序调用这些系统 API 时，R77 会拦截这些请求并修改返回的数据。例如，在查询进程信息时，R77 会过滤掉指定的进程 ID（PID），使得这些进程对外界不可见。

1. **钩住 `NtQuerySystemInformation` 函数：**这是 Windows 系统中获取进程列表的关键函数，R77 会修改该函数的返回值，将被隐藏的进程从进程列表中删除。
2. **修改进程枚举逻辑：**R77 通过动态修改 `NtQuerySystemInformation` 函数的内存空间来操控进程枚举逻辑，从而实现隐藏特定进程。

（2）文件隐藏

行为表现：在运行 R77 后，某些文件和目录在系统的文件管理器中变得不可见。用户可能在文件资源管理器中找不到特定文件，或者通过命令行查看目录时，无法列出这些被隐藏的文件。

技术实现：R77 对文件隐藏的实现类似于进程隐藏，主要通过拦截和修改文件系统的查询请求来进行。

1. **钩住 `NtQueryDirectoryFile` 或 `ZwQueryDirectoryFile`：**这些是 Windows 中用来查询目录内容的函数。R77 会拦截对文件系统的访问请求，并在返回的文件列表中移除被隐藏的文件或目录。
2. **修改查询结果：**通过动态修改文件系统的查询结果，R77 会根据预设规则将某些文件或文件夹从显示结果中移除，使其对用户不可见。
3. **隐藏路径或文件名：**R77 可能会基于文件的路径或文件名来做过滤。通过简单的字符串匹配，R77 会识别需要隐藏的文件或目录，修改 `NtQueryDirectoryFile` 等函数的返回值。

（3）注册表隐藏

行为表现：R77 可以隐藏某些注册表项，使得这些注册表项在注册表编辑器（如 regedit）中不可见。用户无法通过常规的注册表编辑工具访问到这些隐藏的键值。

技术实现：注册表隐藏的原理类似于进程和文件隐藏，都是通过钩住操作系统的注册表相关 API 来修改查询结果。

1. **钩住 NtEnumerateKey 或 NtEnumerateValueKey**：这些是 Windows 中用来枚举注册表键和值的函数。R77 会拦截对注册表的查询请求，过滤掉特定的注册表键或值。
2. **修改注册表遍历逻辑**：R77 修改注册表的枚举函数，以便在查询时跳过某些特定的注册表项，从而使它们对普通用户不可见。
3. **隐藏指定的注册表路径**：通过字符串匹配，R77 可以识别出需要隐藏的注册表路径，并将其从查询结果中去除。

(4) 网络连接隐藏

行为表现：当 R77 被安装在系统中时，特定的网络连接（如恶意进程与远程服务器的通信）可能在网络监控工具中消失。用户使用标准的网络监控工具（如 netstat）时，看不到这些连接。

技术实现：R77 通过修改网络相关的 API 来拦截对网络连接的查询，进而实现隐藏。

1. **钩住 NtDeviceIoControlFile 或类似的网络查询函数**：这些函数用于获取当前的网络连接信息，包括本地和远程 IP 地址、端口等。R77 通过拦截这些函数的调用来修改返回的连接列表。
2. **过滤掉指定的连接信息**：R77 可以通过特定的规则过滤掉某些连接，隐藏恶意进程的网络行为。
3. **修改 Tcpip 栈**：R77 可能还会直接修改 Windows 网络栈，使得某些 TCP/UDP 连接在网络层次上被屏蔽，进而在网络查询工具中看不到这些连接。

(5) 驱动级别操作（更高级别的隐藏）

技术实现：在某些情况下，R77 可能会通过加载和注入内核模式驱动程序来进行更加深层次的隐藏操作。驱动级别的操作可以绕过用户态的监控和限制。

1. **加载内核驱动**：R77 可能会在内核模式下运行，绕过普通的用户态监控工具。内核驱动可以直接访问和修改内核数据结构，如进程列表、文件系统、网络连接等。
2. **通过内核钩子隐藏信息**：内核级的钩子技术可以使得 R77 在操作系统的内核层面上实现对进程、文件、网络连接的透明修改，这些修改即使在用户态工具下也无法被察觉。

三、实验结论及心得体会

心得体会

在本次实验中，基于先前对于R77的使用基础上依照参考文档[Technical Documentation.pdf](#)又进一步分析了R77的执行流程，包括各模块的执行思路；以及其能够实现隐藏各类进程、文件等，驻留，隐蔽性的原因进行了深入探究，对于上次实验中提到的 挂钩API 更加了解了挂钩不同API的不同功能，针对不同的隐藏类型有不同的隐藏方法，例如挂钩不同的API函数或者修改栈等。

实验总结

对 R77 进行深入分析后的简单总结：

1. **高度隐蔽性**：
 - R77 采用了多种技术来确保其在目标系统中保持隐蔽，包括 **进程隐藏**、**文件隐藏**、**注册表项隐藏** 和 **网络连接隐藏** 等。这些技术使得 R77 能够有效地避开大多数传统的防病毒软件和端点检测系统（AV/EDR）。
 - 通过 **内存注入** 和 **shellcode 执行**，R77 避免了通过磁盘文件传播，从而避免了大多数基于文件扫描的安全检测。
2. **内存执行（Fileless Execution）**：

- R77 的安装过程采用了内存执行技术，使用 **shellcode** 在内存中动态加载并执行安装代码，而无需将恶意文件写入硬盘。这使得其在执行时极难被常规的安全扫描工具检测到，避免了基于文件系统的监测。
- 使用 **VirtualProtect** 修改内存权限，并通过函数指针执行 shellcode，是一种典型的 **文件无存储攻击**（fileless attack）技术。

3. 绕过 AMSI (Windows 防恶意软件扫描接口)：

- 通过修改 **amsi.dll** 中的 **AmsiScanBuffer** 函数，R77 可以绕过 Windows 的 AMSI 扫描机制。这种绕过技术不仅可以避免 AMSI 对恶意脚本的检测，而且还可以通过 **多态性** 动态改变恶意代码，进一步增加被检测的难度。

4. DLL 去钩：

- 为了避免 **EDR** 系统的 API 钩子检测，R77 通过加载一个新的 **ntdll.dll** 并恢复原始的系统函数，去除掉 EDR 解决方案注入的钩子。这确保了恶意软件在执行时不会受到钩子的干扰，从而避免被监控和拦截。

5. 自我持久性和自我复制：

- R77 利用修改注册表和创建启动项等方式，确保恶意软件在目标系统上具有 **自我持久性**，即使系统重启，R77 也能继续存在并执行。
- 通过 **进程注入** 和 **注册表修改**，R77 可以在系统中自我复制，确保其在各种恶意行为发生时都能继续存活。

6. 多种攻击向量的结合：

- R77 并不仅仅依赖于单一的攻击向量，它综合使用了内存注入、注册表操作、网络连接隐藏等多种技术，形成了一个全面的攻击框架，使得它能够在不同的防护措施下存活并执行。